

Module 3

Coverage Planning & Analysis in Practice (SV/UVM)

Learning objectives

- Design and implement a practical ****coverage model**** (functional + code), connect it to your UVM env...

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["From Coverage Plan to Concre"]

T2["Functional Coverage Implemen"]

T1 --> T2

T3["Code Coverage Setup and Inte"]

T2 --> T3

Design and implement a practical **coverage model** (functional + code), connect it to your UVM environment and test plan, and perform
firs...

Overview

- Modules 1–2 focused on **what to test** and **how to structure tests and regressions**. Module 3 tu...

Design architecture

1. Coverage in the UVM environment

- Functional coverage lives in ``common_dut/tb/stream_fifo_coverage.sv`` (covergroups, crosses).
- Monitors/agents sample transactions that feed coverpoints (fill level, handshake patterns, errors).
- ``COVERAGE_DESIGN.md`` is the authoritative map from requirements → coverpoints → tests.

Rtl Architecture

```
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
    subgraph DUT["stream_fifo (common_dut/rtl)"]
        SRC["Source interface\ns_valid / s_ready / s_data"]
        MEM["Circular buffer\nmem[DEPTH], wr_ptr, rd_ptr"]
        SNK["Sink interface\nm_valid / m_ready / m_data"]
        STS["Status\nlevel, overflow, underflow"]
    end
```

2. Code coverage integration

- Simulator line/branch/FSM coverage configured in your run scripts (documented in `COVERAGE\_RUN.md`).
- Separate functional vs code coverage roles: intent vs implementation exercised.

```
Verification Architecture  
  
Diagram: Verification Architecture  
(render with mmdc when available)  
flowchart TB  
ENV["UVM env"] --> CG["Functional covergroups\nstream_fifo_coverage.sv"]  
ENV --> CC["Code coverage hooks\n(simulator flags)"]  
PLAN["COVERAGE_DESIGN.md"] --> CG  
RUN["COVERAGE_RUN.md"] --> CC  
CG --> GAP["Gap analysis → new tests"]
```


Coverage model — covergroups

```
covergroup cg_ops;  
    option.per_instance = 1;  
    cp_op_type: coverpoint op_type {  
        bins idle          = { 0 };  
        bins push_only     = { 1 };  
        bins pop_only      = { 2 };  
        bins push_and_pop  = { 3 };  
    }  
endgroup
```

```
//
```

```
-----
```

```
// CG_LEVEL: Occupancy levels (empty / low / mid / high / full)
```

```
//
```

```
-----
```

```
covergroup cg_level:
```

Verification & testing methods

1. Coverage-driven verification (CDV)

- Run a CORE-tier regression subset, merge coverage, and compare against goals in `COVERAGE\_PLAN.md`.
- Use gap analysis to add targeted tests or relax/remove low-value bins — not blind random runs.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

TESTS["Regression tests"] --> MERGE["Coverage merge"]

MERGE --> FUNC["Functional coverage report"]

MERGE --> CODE["Code coverage report"]

FUNC --> GAP["Identify holes / crosses"]

CODE --> GAP

2. Closure loop

- Requirements → tests → coverpoints → measured bins → updated plan/tests.
- ``./scripts/module3.sh --check`` expects covergroup references and coverage design docs.

3. Reporting discipline

- Log each run in `COVERAGE\_RUN.md` with tool, seed, tier, and top uncovered items.

sample() integration point

```
function new(int depth_param = 16);  
    depth = depth_param;  
    cg_ops    = new();  
    cg_level  = new();  
    cg_flags  = new();  
endfunction
```

```
    // Call from monitor on posedge clk after driving level, op_type,  
overflow, underflow  
    function void sample();  
        cg_ops.sample();  
        cg_level.sample();  
        cg_flags.sample();  
    endfunction
```

Syllabus topics

1. From Coverage Plan to Concrete Model

- Refinement of Coverage Goals
- Translating high-level goals into specific covergroups and bins.
- Deciding which metrics are *must-hit* vs *nice-to-have*.
- Coverage Model Structure
- Organizing covergroups by feature, interface, or component.
- Choosing the right placement (agent monitor, scoreboard, reference model, etc.).

2. Functional Coverage Implementation (SV/UVM) (1/2)

- Covergroup Basics
- ``covergroup`` declaration, sampling events (``@``), and options.
- Coverpoints: bins, ranges, wildcards, ignore/illegal bins.
- Cross coverage: when and how to cross (and when not to).
- Sampling Strategy
- Sampling on transactions vs clocks vs protocol events.

2. Functional Coverage Implementation (SV/UVM) (2/2)

- Avoiding oversampling/undersampling.
- Integration with UVM Components
- Embedding covergroups in monitors/scoreboards.
- Using analysis ports/exports for coverage sampling.

3. Code Coverage Setup and Interpretation (1/2)

- Enabling Code Coverage
- Turning on statement/branch/toggle coverage in your simulator.
- Handling compile-time options and run-time switches.
- Interpreting Results
- Reading coverage reports and understanding common metrics.
- Distinguishing real gaps from unreachable or out-of-scope code.

3. Code Coverage Setup and Interpretation (2/2)

- Managing Exclusions
- Identifying truly unreachable code.
- Documenting exclusions and waivers with rationale.

4. Running Coverage-Enabled Regressions (1/2)

- Pilot Regressions
- Start with a modest CORE-tier run (not full regression).
- Ensure coverage data collection is working end-to-end.
- Merging Coverage
- Combining coverage from multiple tests/regressions.
- Understanding how your tool merges coverage across runs and seeds.

4. Running Coverage-Enabled Regressions (2/2)

- Automating Coverage Collection
- Hooks in your regression/CI scripts to generate coverage databases/reports.

5. Coverage Analysis and Gap Closure (1/3)

- Gap Identification
- Finding unhit or low-hit coverpoints and crosses.
- Detecting missing or ineffective coverage (bins that never can be hit).
- Root-Cause Analysis
- Is the gap due to:
- Missing or mis-specified tests?

5. Coverage Analysis and Gap Closure (2/3)

- Overly tight constraints?
- Incomplete coverage model?
- Truly unreachable or out-of-scope functionality?
- Closure Actions
- Adding or modifying tests (directed or constrained-random).
- Adjusting constraints or sequences.

5. Coverage Analysis and Gap Closure (3/3)

- Refining or pruning coverage points.
- Documenting waivers where appropriate.

6. Connecting Coverage to Requirements and Sign-off (1/2)

- Traceability
- Ensuring high-priority requirements are covered by both tests and coverage.
- Updating `REQUIREMENTS\_MATRIX.md` with coverage IDs and status.
- Interpreting Coverage for Sign-off
- Understanding when “100% coverage” is meaningful (and when it isn’t).
- Combining coverage with bug metrics and expert judgment.

6. Connecting Coverage to Requirements and Sign-off (2/2)

- Preparing for Later Modules
- How coverage insights will influence later regression and sign-off modules.

Hands-on examples

Module 3 self-check

```
$ ./scripts/module3.sh --help
```

Terminal: module3_check

```
Module 3: Coverage Planning & Analysis in Practice
```

Usage: ./scripts/module3.sh [OPTIONS]

Module 3: Coverage Planning & Analysis in Practice (SV/UVM)
This script summarizes and checks the coverage artifacts for Module 3.

OPTIONS:

--design-status	Show status of coverage_design.md only
--runs-status	Show status of coverage_runs.md only
--checklist-status	Show status of checklist_module3.md only
--summary	Show all statuses (default)
--check	Media/CI self-check (structure only)
--demo	Print example commands from EXAMPLES.md
--scaffold	Copy templates/*.md into module3/ if missing
--help, -h	Show this help message

EXAMPLES:

```
# Full summary
./scripts/module3.sh

# Focus only on coverage design
./scripts/module3.sh --design-status
```

Exercise scaffold

```
./scripts/module3.sh --scaffold
```

Demo: Coverage design template

```
$ head -30 module3/templates/COVERAGE_DESIGN.md
```

Terminal: ex01_coverage_design

Coverage Design (Module 3)

> ****Purpose****: Define the concrete functional coverage model (covergroups/coverpoints/crosses) a
> ****Module****: 3 – Coverage Planning & Analysis in Practice (SV/UVM)

> ****Instructions****: Fill in this document for your design. Copy from `module3/templates/` if nee

1. Context and Scope

<!-- TODO: DUT name, main interfaces, reference docs. Which features/requirements this coverage

2. Coverage Components and Placement

<!-- TODO: Table of Coverage ID, UVM Component/File, Sampling Event, Notes. Data source and shar

3. Covergroup Specifications

3.1 Operations Coverage

<!-- TODO: Covergroup goal, coverpoints, crosses, requirements covered. -->

3.2 Occupancy / Level Coverage

<!-- TODO: Covergroup goal, coverpoints, crosses, requirements covered. -->

3.3 Backpressure Coverage

<!-- TODO: Covergroup goal, coverpoints, crosses, requirements covered. -->

3.4 Error / Status Coverage

Demo: Coverage run log template

```
$ head -25 module3/templates/COVERAGE_RUN.md
```

Terminal: ex02_coverage_run

Coverage Runs Log (Module 3)

> **Purpose**: Record coverage-enabled regression runs, key metrics, gaps, and follow-up actions
> **Module**: 3 – Coverage Planning & Analysis in Practice (SV/UVM)

> **Instructions**: Fill in this document for your design. Copy from `module3/templates/` if nee

1. Instructions

- Use this file as a **lightweight lab notebook** for coverage work.
- For each significant run, add an entry with:
 - When, what tests, which DUT version, which simulator.
 - Key summary metrics (functional + code).
 - Notable gaps and hypotheses.
 - Planned closure actions.

2. Run Entries

Run 1 – (add title)

<!-- TODO: Date, Engineer, DUT Revision, Environment (simulator, UVM/tool versions). Test Set/Ti

2.1 Functional Coverage Summary (High-Level)

<!-- TODO: Table of Coverage ID, Metric/Notes. -->

Demo: TB coverage skeleton

```
$ head -40 common_dut/tb/stream_fifo_coverage.sv
```

Terminal: ex03_tb_coverage

```
// -----  
// Functional coverage for stream_fifo (Module 3)  
// -----  
//  
// Covergroups aligned with module3/COVERAGE_DESIGN.md.  
// Instantiate this class in stream_fifo_monitor (or equivalent) and call  
// sample() on the appropriate events (e.g., posedge clk).  
//  
`ifndef STREAM_FIFO_COVERAGE_SV  
`define STREAM_FIFO_COVERAGE_SV  
  
class stream_fifo_coverage;  
  
    // Inputs to be set by monitor before sample()  
    bit [31:0] level;    // Current FIFO occupancy (0..DEPTH)  
    bit [1:0] op_type;    // 0=idle, 1=push, 2=pop, 3=push_and_pop  
    bit overflow;  
    bit underflow;  
    int depth;    // DEPTH parameter for bin bounds  
  
    // -----  
    // CG_OPS: Operation types per cycle  
    // -----  
    covergroup cg_ops;  
        option per_instance = 1;  
        cp_op_type: coverpoint op_type {  
            bins idle = { 0 };  
            bins push_only = { 1 };  
            bins pop_only = { 2 };  
            bins push_and_pop = { 3 };  
        }  
    endgroup  
  
    // -----  
    // CG_LEVEL: Occupancy levels (empty / low / mid / high / full)
```

Demo: Reference coverage design

```
$ head -20 module3/.solutions/COVERAGE_DESIGN.md
```

Terminal: ex04_solutions

Coverage Design (Module 3)

> **Purpose**: Define the concrete functional coverage model (covergroups/coverpoints/crosses) a
> **Module**: 3 – Coverage Planning & Analysis in Practice (SV/UVM)

1. Context and Scope

- **DUT**: `stream\_fifo` — parameterizable streaming FIFO with ready/valid handshakes on source
- **Main interfaces / protocols**: Source (push) and sink (pop) ready/valid streaming interfaces
- **Reference docs**:
 - `module1/VERIFICATION\_PLAN.md`
 - `module2/COVERAGE\_PLAN.md`
 - `module1/REQUIREMENTS\_MATRIX.md`

Summarize which **features/requirements** this coverage design targets first:

- **R1** (in-order transfer): Occupancy levels (CG_LEVEL), operation types (CG_OPS), and backpre
- **R2** (overflow flag): Overflow set/clear and levelxoverflow cross (CG_FLAGS, CROSS_LEVEL_FLA
- **R3** (underflow flag): Underflow set/clear and levelxunderflow cross (CG_FLAGS, CROSS_LEVEL_

Demo: Module validation

```
$ ./scripts/module3.sh --check
```

Terminal: ex05_validate

```
[[0;36m=====[[0m
[[0;36mModule 3: Coverage Planning & Analysis in Practice[[0m
[[0;36m=====[[0m
```

Module 3: media self-check
OK: module3/ exists
OK: templates/ exists
OK: EXAMPLES.md exists
OK: docs/MODULE3.md exists
OK: common_dut/rtl/ exists

All required checks passed.

Practice & assessment

What you should know (1/5)

- Design and implement a concrete functional coverage model for your DUT.
- Enable and collect code coverage from your simulator.
- Run coverage-enabled regressions and interpret coverage reports.
- Perform a structured coverage gap analysis and plan closure actions.
- Tie coverage results back to requirements and sign-off criteria.
- Choose 2-4 key features or interfaces.

What you should know (2/5)

- For each, implement:
- At least one covergroup with meaningful coverpoints.
- At least one useful cross (if warranted).
- Integrate sampling into your UVM monitor/scoreboard.
- Verify that bins are being hit in basic tests (e.g., SANITY/CORE tests).
- Enable statement/branch coverage in your simulator build/run scripts.

What you should know (3/5)

- Run a small set of tests (e.g., SANITY or a subset of CORE).
- Generate and inspect the coverage report.
- Identify:
 - Any obvious coverage gaps in well-understood code.
 - Any code that appears to be unreachable or out-of-scope.
- In ``module3/COVERAGE_RUN.md``, record at least one coverage run:

What you should know (4/5)

- Date, DUT version, test set/tier, seed policy, simulator version.
- Key coverage metrics (summary) and major observations.
- List at least 5 concrete coverage gaps and your initial hypotheses.
- For at least 3 significant gaps:
- Decide whether to: add tests, tune constraints, refine coverage, or waive.
- Document the chosen action and expected impact.

What you should know (5/5)

- Update:
- `module2/TEST\_PLAN.md` (new/updated tests).
- `module2/COVERAGE\_PLAN.md` /
 `module3/COVERAGE\_DESIGN.md`.
- `module1/REQUIREMENTS\_MATRIX.md` (coverage mappings, waivers).

Exercises

- Over-Coverage vs Under-Coverage
- Coverage vs Bug History (if available)
- Tooling Experiment

Assessment checklist

- Functional coverage model implemented for key features/interfaces.
- Code coverage successfully enabled and reports generated.
- At least one coverage run documented in ``COVERAGE_RUN.md``.
- Coverage gaps identified and categorized (test/constraint/model/waiver).
- Closure actions planned and reflected in test/coverage/requirements docs.

Summary & next steps

- Design and implement a practical ****coverage model**** (functional + code), connect it to your UVM env...
- Complete module3/CHECKLIST.md
- Review module3/EXAMPLES.md and run each lab
- Next: UVM Environment and Checker Maturity